

LECTURE 04

DOM II, *events.*

Clicks, keys, forms, bubbling, and the pattern that keeps interactive pages from becoming spaghetti.

listeners

bubbling

target

delegation

forms

PRETTIER

Let the editor clean the room.

VS CODE SETUP

- ✓ Extensions → search Prettier
- ✓ Settings → Format On Save → On
- ✓ Save files and stop manually lining up braces

DOM event code nests quickly: callbacks inside listeners, conditionals inside callbacks, handlers shared across elements.

Auto-formatting keeps the code readable while your attention stays on behavior.

formatting is not the lesson. it clears space for the lesson.

BROWSER SIGNALS

Interesting things fire events.

CLICK

User clicks an element.

MOUSEOVER

Mouse enters an element.

SUBMIT

A form is submitted.

CHANGE

An input value changes.

SCROLL

User scrolls the page.

KEYUP / KEYDOWN

A keyboard key moves.

EVENT OBJECT

Every handler gets context.

```
button.addEventListener("click", (event) => {  
  console.log(event.type)  
  console.log(event.target)  
  console.log(event.clientX, event.clientY)  
})
```

COMMON PROPERTIES

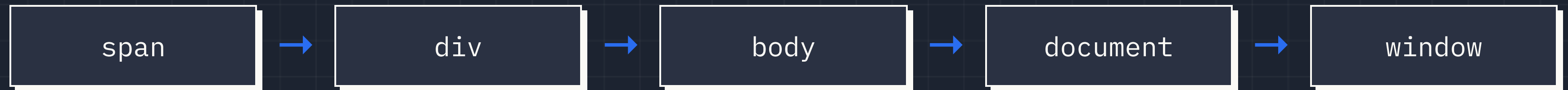
- ✓ type - event name
- ✓ target - element that triggered it
- ✓ timeStamp - when it happened
- ✓ key - keyboard key
- ✓ clientX / clientY - mouse position

event.target answers: what did the user actually touch?

BUBBLING

Events climb the tree.

When an event fires on a node, it bubbles upward through every ancestor until it reaches window.



ORDER MATTERS

The target fires first. Parents hear it next.

```
const div = document
  .getElementsByName("div")[0]
div.addEventListener("click", () =>
  console.log("div clicked")
)

const span = document
  .getElementsByName("span")[0]
span.addEventListener("click", () =>
  console.log("span clicked")
)

// click span: span clicked, then div clicked
```

If a span lives inside a div, a click on the span is also a click inside the div.

The browser gives each ancestor a chance to respond.

bubbling is not a bug. it is the model.

ESCAPE HATCH

Sometimes the event should stop climbing.

```
element.addEventListener("click", (event) => {  
  event.stopPropagation()  
  
  // parent listeners will not run  
})
```

`stopPropagation()` prevents the event from bubbling further.

Use it when parent behavior would be wrong: nested buttons, menus, overlays, or dismissible UI.

reach for it deliberately. bubbling is useful.

THE PATTERN

One parent listener. Many child actions.

```
const div = document.querySelector("div")

div.addEventListener("click", (event) => {
  // event.target = what was clicked
  // event.currentTarget = listener owner

  if (event.target.tagName === "BUTTON") {
    console.log(
      "A button was clicked:",
      event.target.textContent
    )
  }
})
```

Delegation takes advantage of bubbling.

- ✓ One listener instead of N listeners.
- ✓ Works for children added later.
- ✓ Centralizes behavior for a region of UI.

TARGET VS CURRENTTARGET

Two elements matter.

`EVENT.TARGET`

The actual trigger.

The deepest element the user interacted with: maybe a button, icon, span, or input.

`EVENT.CURRENTTARGET`

The listener owner.

The element where the event listener is attached. In delegation, often the parent.

target asks "what got clicked?" currentTarget asks "who is listening?"

QUICK REFERENCE

The DOM event toolbelt.

```
// Search
document.getElementById("id")
document.querySelector(".class")
document.querySelectorAll("div")

// Traverse
el.parentElement
el.children
el.nextElementSibling

// Modify
el.style.color = "red"
el.innerText = "new text"
```

```
// Events
el.addEventListener("click", (e) => {
  // ...
})

e.target          // actual trigger
e.currentTarget    // listener owner

e.stopPropagation()
e.preventDefault()
```